

Conversão dos Programas Iterativos W1 e W2 para Programas Recursivos

Notação usada: P é E0 onde R1 def E1, R2 def E2, ...

Neste documento, primeiro são dadas as conversões de W1 e W2 para a forma recursiva, usando a notação formal do material. Depois é mostrada a expansão passo a passo das sub-rotinas, para deixar claro como a recursão simula o comportamento iterativo.

Traduza os programas iterativos W_1 e W_2 definidos na Figura e na Figura para programas recursivos.

Programa Iterativo W_1
enquanto T faça (F;(se T faça $\checkmark$ senão G))

Figura 2.31 Programa iterativo W_1

Programa Iterativo W_2
enquanto T faça (F;enquanto T faça (F);G)

Figura 2.32 Programa iterativo W_2

1. Programa Iterativo W1

Programa iterativo dado:

```
enquanto T
faça (F; (se T faça  $\checkmark$  senão G))
```

Programa recursivo correspondente:

```
P é R onde
R def (se T então F;S;R senão  $\checkmark$ ),
S def (se T então  $\checkmark$  senão G)
```

Leitura da solução:

- A sub-rotina R representa o laço externo “enquanto T faça ...”.
- Se T é falso no início de R, a execução termina com $\checkmark$.
- Se T é verdadeiro, executa-se F, depois S, e então R é chamada novamente.
- A sub-rotina S representa exatamente a parte interna “se T faça $\checkmark$ senão G”.

Expansão passo a passo de W1:

Começando pela expressão inicial:

P é R

Primeira expansão de R:

R
⇒ (se T então F;S;R senão √)

Caso 1: T é falso logo na entrada.

R
⇒ (se T então F;S;R senão √)
⇒ √

Nesse caso, o laço while não executa nenhuma iteração, o que coincide exatamente com o comportamento de “enquanto T faça ...”.

Caso 2: T é verdadeiro na entrada.

R
⇒ (se T então F;S;R senão √)
⇒ F;S;R

Agora expande-se S:

S
⇒ (se T então √ senão G)

Subcaso 2.1: depois de F, T é verdadeiro.

F;S;R
⇒ F;(se T então √ senão G);R
⇒ F;√;R
⇒ F;R

Ou seja, nessa iteração o programa executa F e, como o segundo teste deu verdadeiro, não executa G. Depois volta ao topo do laço por meio da chamada recursiva R.

Subcaso 2.2: depois de F, T é falso.

F;S;R
⇒ F;(se T então √ senão G);R
⇒ F;G;R

Nesse caso, a iteração executa F e depois G, e então volta ao topo do laço pela chamada recursiva R.

Resumo intuitivo de W1:

- R controla a repetição externa.
- S controla a decisão interna entre fazer nada (√) ou executar G.
- A nova chamada de R simula a próxima iteração do enquanto.

2. Programa Iterativo W2

Programa iterativo dado:

```
enquanto T
  faça (F; enquanto T faça (F); G)
```

Programa recursivo correspondente:

```
P é R onde
R def (se T então F;S;G;R senão √) ,
S def (se T então F;S senão √)
```

Ou outra forma correta:

```
P é R onde
R def (se T então F;S senão √)
S def (se T então F;S senão G;R)
```

Leitura da solução:

- A sub-rotina R representa o laço externo.
- A sub-rotina S representa o laço interno “enquanto T faça (F)”.
- Se T é falso em R, o programa termina.
- Se T é verdadeiro em R, o programa executa F, entra no laço interno S, executa G e volta a R.

Expansão passo a passo de W2:

Começando pela expressão inicial:

P é R

Primeira expansão de R:

```
R
⇒ (se T então F;S;G;R senão √)
```

Caso 1: T é falso logo na entrada.

```
R
⇒ (se T então F;S;G;R senão √)
⇒ √
```

Nesse caso, o while externo não executa nenhuma iteração.

Caso 2: T é verdadeiro na entrada.

R

$\Rightarrow$ (se T então F;S;G;R senão $\surd$)

$\Rightarrow$ F;S;G;R

Agora expandimos S, que representa o while interno:

S

$\Rightarrow$ (se T então F;S senão $\surd$)

Subcaso 2.1: T é falso na primeira avaliação de S.

F;S;G;R

$\Rightarrow$ F; (se T então F;S senão $\surd$);G;R

$\Rightarrow$ F; $\surd$;G;R

$\Rightarrow$ F;G;R

Isso significa: o laço externo executou seu primeiro F; o laço interno não executou nenhuma repetição; depois executa-se G; por fim volta-se ao laço externo com R.

Subcaso 2.2: T é verdadeiro em S pelo menos uma vez.

F;S;G;R

$\Rightarrow$ F; (se T então F;S senão $\surd$);G;R

$\Rightarrow$ F;F;S;G;R

Agora o laço interno continua, porque S reaparece. Enquanto T permanecer verdadeiro, novos F serão executados dentro de S.

Se o teste interno continuar verdadeiro mais uma vez, temos:

F;F;S;G;R

$\Rightarrow$ F;F; (se T então F;S senão $\surd$);G;R

$\Rightarrow$ F;F;F;S;G;R

Quando finalmente T ficar falso em S, o laço interno termina:

F;F;F;S;G;R

$\Rightarrow$ F;F;F; $\surd$;G;R

$\Rightarrow$ F;F;F;G;R

Depois disso, G é executado e o controle volta ao laço externo por meio da chamada recursiva R.

Resumo intuitivo de W2:

- R controla o while externo.
- S controla o while interno.
- A repetição interna aparece porque S se chama novamente enquanto T for verdadeiro.
- Quando S termina, G é executado e a computação retorna ao ciclo externo com R.

3. Conclusão geral

Nos dois casos, a recursão substitui explicitamente a repetição. Em W1, há um único laço externo e uma decisão interna simples; por isso bastam as sub-rotinas R e S. Em W2, há um laço externo e um laço interno; por isso R modela o externo e S modela o interno.

A operação vazia $\surd$ aparece como o ponto em que um ramo termina sem executar nova operação. No formalismo do material, isso é o que permite encerrar corretamente a expansão recursiva daquele ramo.